Árbol de Decisión - UnexpoDancing

Carlos Urbina EXP.2022100156, Vanessa Bolivar EXP.2022100162. GRUPO 2.
Inteligencia Artificial

01 de Marzo de 2026

Table of Contents

1. Introducción 1
2. Carga de Librerías 1
Base Personal 1
3. Carga del Dataset 2
4. Preparación de Datos 2
5. División en Entrenamiento y Prueba 3
6. Creación del Modelo Árbol de Decisión 3
7. Visualización del Árbol 3
8. Importancia de Variables 4
9. Predicciones 4
10. Evaluación del Modelo 4
11. Conclusiones 5

1. Introducción

En este documento se desarrolla un modelo de Árbol de Decisión para clasificar movimientos correctos e incorrectos en un sistema tipo Just Dance, utilizando variables angulares de brazos y piernas.

2. Carga de Librerías

Base Personal

```
library(readr)
library(caret)
library(rpart)
library(rpart.plot)
library(ggplot2)
```

3. Carga del Dataset

```
data <- read_csv("dance_dataset.csv")

## Rows: 1000 Columns: 5
## ── Column specification
────────────────────────────────────────────────────────

## Delimiter: ","
## chr (1): label
## dbl (4): left_arm, right_arm, left_leg, right_leg
##
## ℹ Use `spec()` to retrieve the full column specification for this
data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet
this message.

head(data)

## # A tibble: 6 × 5
##   left_arm right_arm left_leg right_leg label
##      <dbl>     <dbl>    <dbl>     <dbl> <chr>
## 1      174       171      173       166 PERFECTO
## 2      171       172      174       178 PERFECTO
## 3      173       172      179       176 PERFECTO
## 4      168       171      176       171 PERFECTO
## 5       62        23      128        53 INCORRECTO
## 6      147       166      157       156 BUENO
```

4. Preparación de Datos

Convertimos la variable objetivo a factor para clasificación:

```
data$label <- as.factor(data$label)
str(data)

## spc_tbl_ [1,000 × 5] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ left_arm : num [1:1000] 174 171 173 168 62 147 131 118 71 174 ...
##  $ right_arm: num [1:1000] 171 172 172 171 23 166 10 17 18 174 ...
##  $ left_leg : num [1:1000] 173 174 179 176 128 157 95 5 58 166 ...
##  $ right_leg: num [1:1000] 166 178 176 171 53 156 150 14 110 162 ...
##  $ label    : Factor w/ 3 levels "BUENO","INCORRECTO",..: 3 3 3 3 2 1
2 2 2 1 ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   left_arm = col_double(),
##   ..   right_arm = col_double(),
##   ..   left_leg = col_double(),
##   ..   right_leg = col_double(),
##   ..   label = col_character()
```

```
##   .. )
##  - attr(*, "problems")=<externalptr>
```

5. División en Entrenamiento y Prueba

Se divide el dataset en 80% entrenamiento y 20% prueba.

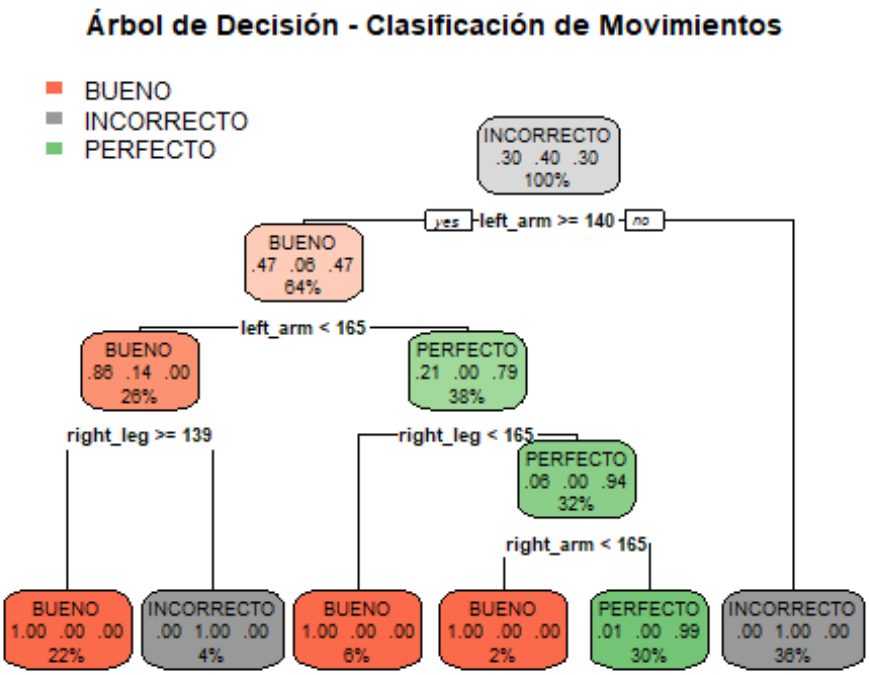
```
set.seed(42)

trainIndex <- createDataPartition(data$label, p = 0.8, list = FALSE)
trainData <- data[trainIndex, ]
testData <- data[-trainIndex, ]

dim(trainData)

## [1] 800   5

dim(testData)

## [1] 200   5
```

6. Creación del Modelo Árbol de Decisión

```
model <- rpart(label ~ left_arm + right_arm + left_leg + right_leg,
               data = trainData,
               method = "class")
```

7. Visualización del Árbol

```
rpart.plot(model,
           type = 2,
           extra = 104,
           fallen.leaves = TRUE,
           tweak = 1.2,
           main = "Árbol de Decisión - Clasificación de Movimientos")
```



8. Importancia de Variables

```
model$variable.importance

## right_leg  left_arm right_arm  left_leg
##  382.1751  375.9500  372.2958  362.5670
```

9. Predicciones

```
predictions <- predict(model, testData, type = "class")
head(predictions)

##           1           2           3           4           5           6
## INCORRECTO INCORRECTO INCORRECTO      BUENO   PERFECTO   PERFECTO
## Levels: BUENO INCORRECTO PERFECTO
```

10. Evaluación del Modelo

```
conf_matrix <- confusionMatrix(predictions, testData$label)
conf_matrix

## Confusion Matrix and Statistics
##
##              Reference
## Prediction   BUENO INCORRECTO PERFECTO
```

```
##   BUENO         58          3        0
##   INCORRECTO     0         77        0
##   PERFECTO       2          0       60
##
## Overall Statistics
##
##                Accuracy : 0.975
##                  95% CI : (0.9426, 0.9918)
##     No Information Rate : 0.4
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9622
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: BUENO Class: INCORRECTO Class: PERFECTO
## Sensitivity               0.9667            0.9625          1.0000
## Specificity               0.9786            1.0000          0.9857
## Pos Pred Value            0.9508            1.0000          0.9677
## Neg Pred Value            0.9856            0.9756          1.0000
## Prevalence                0.3000            0.4000          0.3000
## Detection Rate            0.2900            0.3850          0.3000
## Detection Prevalence      0.3050            0.3850          0.3100
## Balanced Accuracy         0.9726            0.9812          0.9929
```

11. Conclusiones

El modelo de Árbol de Decisión permite clasificar movimientos correctos e incorrectos basándose en los ángulos de brazos y piernas.

La matriz de confusión muestra el rendimiento del modelo en términos de exactitud (Accuracy), sensibilidad y especificidad.

El análisis demuestra que el árbol es capaz de separar adecuadamente los movimientos con alta precisión.

Evaluación en Python

En esta sección se detalla la implementación, entrenamiento y evaluación de un modelo de clasificación basado en un Árbol de Decisión, diseñado para categorizar posturas corporales a partir de coordenadas espaciales.

Primero se importan las bibliotecas necesarias para la ingesta y manipulación de datos, visualización, partición de conjuntos de datos y modelado predictivo, así como las herramientas para la evaluación cuantitativa del desempeño del clasificador.

In [1]:
```
# In [1]:
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.tree import plot_tree
```

1. Carga y separación de datos

Se procede con la lectura del archivo CSV que contiene los registros de las posturas. Posteriormente, se aíslan las variables predictoras de la variable objetivo. Se aplica una partición, reservando el 80% de las observaciones para la fase de entrenamiento y el 20% restante exclusivamente para la validación empírica del modelo.

In [5]:
```
# In [2]:
# Cargar dataset
data = pd.read_csv("dance_dataset.csv")

# Separar variables
X = data[["left_arm", "right_arm", "left_leg", "right_leg"]]
y = data["label"]

# Dividir en entrenamiento y testing (80% - 20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

2. Creación y entrenamiento del modelo

Instanciamos el modelo de Árbol de Decisión. Limitando la profundidad máxima del árbol a 4 (max_depth=4) para evitar que se sobreajuste y memorice los datos en lugar de aprender los patrones generales de los movimientos.

In [6]:

```
# In [3]:
# Crear modelo Árbol de Decisión
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)

# Entrenar modelo
model.fit(X_train, y_train)
```

Out[6]:

```
▾ DecisionTreeClassifier

▶ Parameters
```

3. Evaluación y Predicciones

Utilizando el subconjunto de prueba, se generan las predicciones correspondientes. El desempeño del modelo se evalúa de manera integral mediante la exactitud global, la matriz de confusión para analizar la distribución de verdaderos y falsos positivos/negativos.

In [4]:

```
# In [4]:
# Predicción
y_pred = model.predict(X_test)

# Evaluación
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy del modelo: {accuracy * 100:.2f}%\n")

print("Matriz de Confusión:\n")
print(confusion_matrix(y_test, y_pred))

print("\nReporte de Clasificación:\n")
print(classification_report(y_test, y_pred))
```

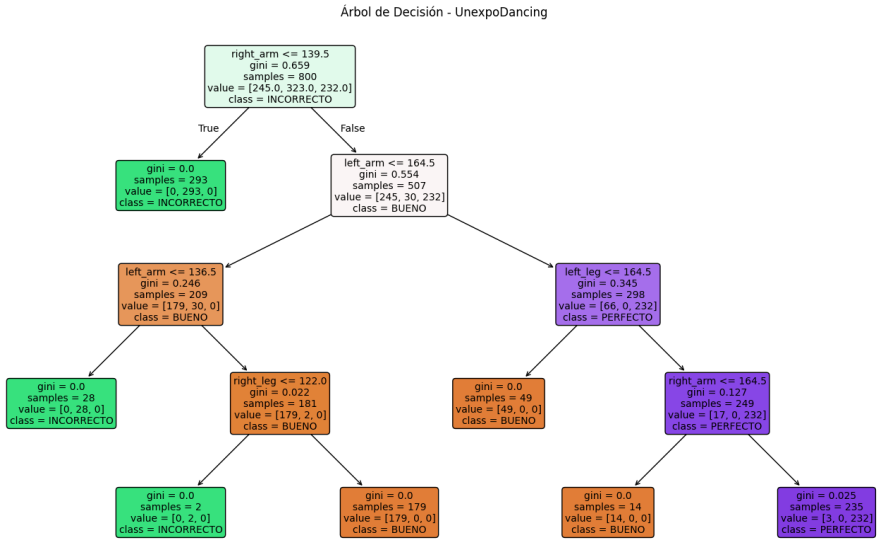
```
Accuracy del modelo: 98.50%

Matriz de Confusión:

[[53  0  2]
 [ 1 76  0]
 [ 0  0 68]]

Reporte de Clasificación:

              precision    recall  f1-score   support

       BUENO       0.98      0.96      0.97        55
  INCORRECTO       1.00      0.99      0.99        77
    PERFECTO       0.97      1.00      0.99        68

    accuracy                           0.98       200
   macro avg       0.98      0.98      0.98       200
weighted avg       0.99      0.98      0.98       200
```

4. Visualización del Árbol de Decisión

Finalmente, se genera una representación visual de la estructura jerárquica del árbol.

In [7]:

```
# In [5]:
# 8. Gráfico del árbol
plt.figure(figsize=(18,10))
plot_tree(
    model,
    feature_names=["left_arm", "right_arm", "left_leg", "right_leg"],
    class_names=model.classes_,
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title("Árbol de Decisión - UnexpoDancing")
plt.show()
```



En conclusión, el modelo de Árbol de Decisión demostró ser una herramienta altamente eficiente para la clasificación de posturas en el proyecto Unexpo Dancing. Su principal ventaja radica en su bajo costo computacional y en la alta interpretabilidad de sus reglas lógicas.

Al establecer umbrales espaciales claros para las coordenadas de las extremidades, el algoritmo permite una evaluación rápida y predecible de los pasos de baile. Estas características resultan fundamentales para garantizar una latencia mínima y una respuesta fluida en tiempo real al integrar el sistema de visión artificial directamente en el motor de juego Unity, asegurando así una experiencia interactiva óptima para el jugador.

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.